

```

#include <stdio.h>
#include <stdint.h>
#include <string.h>

// ===== RV-SVE ISA 정의 =====
#define OPCODE_RV_SVE 0b1011011
#define FUNCT3_VMUL_VV 0b010
#define FUNCT3_VLOAD_VA 0b100
#define FUNCT3_VSTORE_AV 0b101
#define FUNCT7_VECTOR 0b0000000

// ===== 명령어 구조 =====
typedef struct {
    uint64_t raw_inst;
    int rd, rs1, rs2;
    int funct3;
    int is_vector;
    char opcode[16];
} rv_sve_uop;

// ===== 세그먼트 컨텍스트 =====
typedef struct {
    rv_sve_uop uop;
    int predictedLoad;
    char priority[8];
    char dispatchToken[32];
} GPUContext;

// ===== Core 0: evaluate_motive =====
GPUContext evaluate_motive(uint64_t raw) {
    GPUContext ctx;
    ctx.uop.raw_inst = raw;
    ctx.uop.rd = (raw >> 7) & 0x1F;
    ctx.uop.rs1 = (raw >> 15) & 0x1F;
    ctx.uop.rs2 = (raw >> 20) & 0x1F;
    ctx.uop.funct3 = (raw >> 12) & 0x7;
    ctx.uop.is_vector = ((raw & 0x7F) == OPCODE_RV_SVE);

    if (ctx.uop.is_vector) {
        if (ctx.uop.funct3 == FUNCT3_VMUL_VV) strcpy(ctx.uop.opcode, "VMUL.VV");
        else if (ctx.uop.funct3 == FUNCT3_VLOAD_VA) strcpy(ctx.uop.opcode, "VLOAD.VA");
        else if (ctx.uop.funct3 == FUNCT3_VSTORE_AV) strcpy(ctx.uop.opcode,
"VSTORE.AV");
        else strcpy(ctx.uop.opcode, "VOP");
    } else {
        strcpy(ctx.uop.opcode, "NOP");
    }
    return ctx;
}

```

```

}

// ===== Core 1: render_if_needed =====
GPUContext render_if_needed(GPUContext ctx) {
    if (strcmp(ctx.uop.opcode, "VLOAD.VA") == 0) {
        printf("[Core1] Render triggered by VLOAD.VA\n");
    }
    return ctx;
}

// ===== Core 2: simulate_outcome =====
GPUContext simulate_outcome(GPUContext ctx) {
    ctx.predictedLoad = (ctx.uop.rs1 + ctx.uop.rs2) * 64;
    printf("[Core2] Predicted Load: %d\n", ctx.predictedLoad);
    return ctx;
}

// ===== Core 3: importance_rank =====
GPUContext importance_rank(GPUContext ctx) {
    strcpy(ctx.priority, ctx.predictedLoad > 4096 ? "high" : "low");
    printf("[Core3] Priority: %s\n", ctx.priority);
    return ctx;
}

// ===== Core 4: dispatch_policy =====
GPUContext dispatch_policy(GPUContext ctx) {
    if (strcmp(ctx.priority, "high") == 0)
        sprintf(ctx.dispatchToken, "DISPATCH_%s_FAST", ctx.uop.opcode);
    else
        sprintf(ctx.dispatchToken, "DISPATCH_%s_IDLE", ctx.uop.opcode);
    printf("[Core4] Token: %s\n", ctx.dispatchToken);
    return ctx;
}

// ===== Core 5: trace_context =====
GPUContext trace_context(GPUContext ctx) {
    printf("[Core5] Trace: %s r%d ← r%d, r%d\n", ctx.uop.opcode, ctx.uop.rd, ctx.uop.rs1,
    ctx.uop.rs2);
    return ctx;
}

// ===== Core 6: defer_to_idle_render_thread =====
GPUContext defer_to_idle_render_thread(GPUContext ctx) {
    if (strcmp(ctx.priority, "low") == 0)
        printf("[Core6] Deferred to idle thread: %s\n", ctx.dispatchToken);
    return ctx;
}

```

```

// ===== Core 7: correction_core =====
GPUContext correction_core(GPUContext ctx) {
    printf("[Core7] Correction: IPC synchronized for %s\n", ctx.uop.opcode);
    return ctx;
}

// ===== GPU 실행 파이프라인 =====
void run_gpu_pipeline(uint64_t raw_inst) {
    GPUContext ctx = evaluate_motive(raw_inst);
    ctx = render_if_needed(ctx);
    ctx = simulate_outcome(ctx);
    ctx = importance_rank(ctx);
    ctx = dispatch_policy(ctx);
    ctx = trace_context(ctx);
    ctx = defer_to_idle_render_thread(ctx);
    ctx = correction_core(ctx);
}

// ===== 테스트 =====
int main() {
    // 예: VMUL.VV 명령어 생성
    uint64_t raw = ((uint64_t)FUNCT7_VECTOR << 25) | ((uint64_t)2 << 20) |
        ((uint64_t)1 << 15) | ((uint64_t)FUNCT3_VMUL_VV << 12) |
        ((uint64_t)0 << 7) | ((uint64_t)OPCODE_RV_SVE);
    run_gpu_pipeline(raw);
    return 0;
}

```